

4주차 과제

파이썬 머신러닝 완벽 가이드 4장 (4.5장~4.8, 4.10~4.11) 개념정리 및 필사

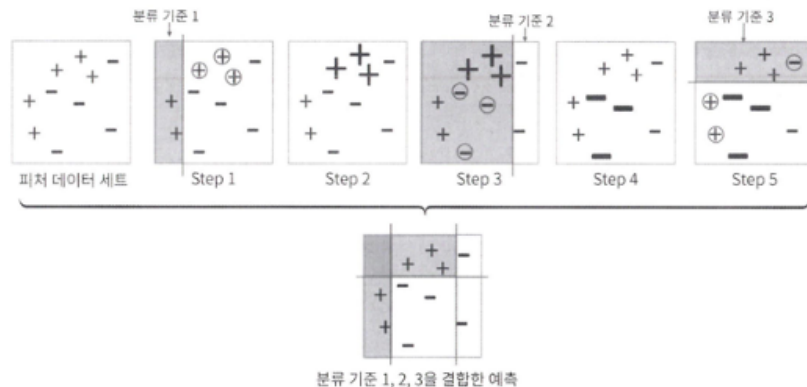
4장 분류

4.5 GBM(Gradient Boosting Machine)

GBM의 개요 및 실습

- 부스팅 알고리즘: 여러 개의 약한 학습기를 순차적으로 학습-예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식 cf. 대표 구현: AdaBoost(Adaptive boosting)와 그래디언트 부스트

▼ AdaBoost의 학습 진행 과정



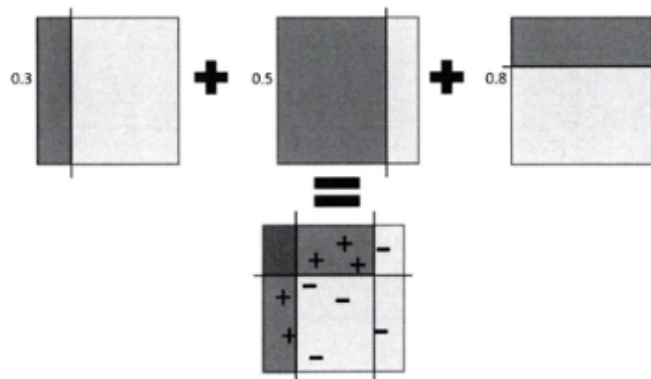
- Step1: 첫 번째 약한 학습기(weak learner)가 분류 기준 1로 +/-를 분류한 것. 동그라미로 표시된 + 데이터는 + 데이터가 잘못 분류된 오류 데이터
- Step2: 이 오류 데이터에 대해 가중치 부여. 가중치 부여된 오류 + 데이터는 다음 약한 학습기가 더 잘 분류할 수 있게 크기 커짐
- Step3: 두 번째 약한 학습기가 분류 기준 2로 +/- 분류. 마찬가지로 동그라미로 표시된 - 데이터는 잘못 분류된 오류 데이터
- Step4: 잘못 분류된 이 - 오류 데이터에 대해 다음 약한 학습기가 잘 분류할 수 있게 더 큰 가중치 부여 (오류 - 데이터의 크기 증가)

- Step5: 세 번째 약한 학습기가 분류 기준 3으로 +/- 분류, 오류 데이터 찾음. 에이다 부스트는 이렇게 약한 학습기가 순차적으로 오류 값에 대해 가중치를 부여한 예측 결정 기준을 모두 결합해 예측 수행
- 마지막으로 맨 아래에는 첫 번째, 두 번째, 세 번째 약한 학습기를 모두 결합한 결과 예측, 개별 약한 학습기보다 훨씬 정확도가 높아졌음

▼ 가중치 부여 과정

개별 약한 학습기는 가중치를 부여해 결합!

ex. 첫 번째 학습기에 가중치 0.3, 두 번째 학습기에 가중치 0.5, 세 번째 학습기에 가중치 0.8 부여 ⇒ 모두 결합해 예측 수행



GBM은 에이다부스트와 유사하나, 가중치 업데이트를 경사 하강법을 이용하는 것이 큰 차이. → 오류 값은 실제 값 - 예측값.

y: 분류의 실제 결과값 / $x_1, x_2, x_3, \dots, x_n$: 피쳐 / $F(x)$: 피쳐에 기반한 예측 함수

$$\text{오차값} = y - F(x)$$

$$h(x) = y - F(x)$$

GBM 하이퍼 파라미터 및 튜닝

- loss: 경사 하강법에서 사용할 비용 함수 지정. 특별한 이유가 없으면 기본값인 'deviance'를 그대로 적용
- learning_rate: GBM이 학습을 진행할 때마다 적용하는 학습률. Weak learner가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수. 0~1 사이의 값을 지정할 수 있으며 기본값: 0.1 너무 작은 값을 적용시, 업데이트 되는 값이 작아져 최소 오류 값을 찾아 예측 성능이 높아질 가능성 높음. but, 많은 weak learner는 순차적인 반복이 필요해서 수행 시간이 오래 걸리고, 또 너무 작게 설정하면 모든 weak learner의 반복이 완료돼도 최소 오류 값 찾지 x (반대로 큰 값 정공시 최소 오류 값 찾지 x, 예측 성능 감소&빠른 수행 가능성 높아짐!)

⇒ learning_rate ↔ n_estimators (상호 보완적으로 조합해 사용)

- n_estimators: weak learner의 개수. weak learner가 순차적으로 오류를 보정 → 개수가 많을수록 예측 성능이 일정 수준까지는 좋아질 수 O but, 개수가 많을수록 수행 시간 증가. 기본값: 100
- subsample: weak learner가 학습에 사용하는 데이터의 샘플링 비율. 기본값: 1 → 전체 학습 데이터를 기반으로 학습한다는 의미. (과적합 염려시 subsample을 1보다 작은 값으로 설정)

4.6 XGBoost(eXtra Gradient Boost)

XGBoost 개요

: 분류에 있어 일반적으로 다른 머신러닝보다 뛰어난 예측 성능을 나타냄.

XGBoost는 GBM에 기반하지만, GBM 단점(느린 수행 시간 및 과적합 규제 부재 등)을 해결. 특히 병렬 CPU 환경에서 병렬 학습이 가능 → 기존 GBM보다 빠르게 학습 완료 가능

▼ XGBoost의 주요 장점

항목	설명
뛰어난 예측 성능	일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘합니다.
GBM 대비 빠른 수행 시간	일반적인 GBM은 순차적으로 Weak learner가 가중치를 증감하는 방법으로 학습하기 때문에 전반적으로 속도가 느립니다. 하지만 XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장합니다. 아쉽게도 XGBoost가 일반적인 GBM에 비해 수행 시간이 빠르다는 것이지, 다른 머신러닝 알고리즘(예를 들어 랜덤 포레스트)에 비해서 빠르다는 의미는 아닙니다.
항목	설명
과적합 규제 (Regularization)	표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 좀 더 강한 내구성을 가질 수 있습니다.
Tree pruning (나무 가지치기)	일반적으로 GBM은 분할 시 부정 손실이 발생하면 분할을 더 이상 수행하지 않지만, 이러한 방식도 자칫 지나치게 많은 분할을 발생시킬 수 있습니다. 다른 GBM과 마찬가지로 XGBoost도 max_depth 파라미터로 분할 깊이를 조정하기도 하지만, tree pruning으로 더 이상 긍정 이익이 없는 분할을 가지치기 해서 분할 수를 더 줄이는 추가적인 장점을 가지고 있습니다.
자체 내장된 교차 검증	XGBoost는 반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증을 수행해 최적화된 반복 수행 횟수를 가질 수 있습니다. 지정된 반복 횟수가 아니라 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복을 중간에 멈출 수 있는 조기 중단 기능이 있습니다.
결손값 자체 처리	XGBoost는 결손값을 자체 처리할 수 있는 기능을 가지고 있습니다.

- XGBoost의 핵심 라이브러리는 C/C++ 로 작성
- 파이썬 패키지명: "xgboost"; 패키지 내에 XGBoost 전용의 파이썬 패키지&사이킷런과 호환되는 래퍼용 XGBoost가 함께 존재

파이썬 래퍼 XGBoost 모듈

초기의 독자적인 XGBoost 프레임워크 기반

사이킷런 래퍼 XGBoost 모듈

사이킷런과 연동되는 모듈

파이썬 래퍼 XGBoost 하이퍼 파라미터

XGBoost는 GBM과 유사한 하이퍼 파라미터를 동일하게 가짐 + 조기 중단+하이퍼 파라미터 등 추가



파이썬 래퍼 XGBoost 하이퍼 파라미터의 유형별 분류

- **일반 파라미터**: 일반저금로 실행 시 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터로서 디폴트 파라미터 값을 바꾸는 경우 거의 X
- **부스터 파라미터**<대부분의 하이퍼 파라미터>: 트리 최적화, 부스팅, regularization 등과 관련된 파라미터 등 지칭
- **학습 태스크 파라미터**: 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터

주요 파라미터

주요 일반 파라미터

- ▼ **booster**
gbtree(tree based model) or gblinear(linear model) 선택. 디폴트=gbtree
- ▼ **silent**
디폴트=0, 출력 메시지 x = 1
- ▼ **nthread**
CPU의 실행 스레드 개수 조정 → 디폴트는 CPU의 전체 스레드를 다 사용하는 것. 멀티 코어/스레드

주요 부스터 파라미터

- ▼ **eta[default=0.3, alias: learning_rate]**
GBM의 학습률과 같은 파라미터. 0~1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값
- ▼ **num_boost_rounds**
GBM의 n_estimators와 같은 파라미터

학습 태스크 파라미터

- ▼ **objective**
최소값을 가져야 할 손실 함수 정의
- ▼ **binary:logistic**
이진 분류일 때 적용
- ▼ **multi:softmax**
다중 분류일 때 적용
- ▼ **multi:softprob**
개별 레이블 클래스의 해당되는 예측 확률 반환

CPU 시스템에서
전체 CPU사용
X → 일부만 사용

`min_child_weight[de`
트리에서 추가적으
로 가치를 나눌지
결정하기 위해 필요
한 데이터들의
weight 총합. (과
적합 조절)

▼ `eval_metric`
검증에 사용되는 합
수 정의
회귀: rmse
분류: error

▼
`gamma[default=0,`
`alias:`
`min_split_loss]`
트리의 리프 노드를
추가적으로 나눌지
결정할 최소 손실
감소 값 (값이 클수
록 과적합 감소 효
과)

▼
`max_depth[default=`
0 지정시 깊이에 제
한 X / 3~10 값 지
정

▼
`sub_sample[default=`
트리가 커져서 과적
합되는 것을 제어하
기 위해 데이터 샘
플링하는 비율을 지
정.

▼
`colsample_bytree[de`
트리 생성에 필요한
피쳐(칼럼)를 임의
로 샘플링 하는 데
사용. (매우 많은 피

처가 있는 경우 과
적합 조정)

▼ `lambda[default=1,
alias: reg_lambda]`

피쳐 개수가 많을
경우, 적용 검토 →
값이 클수록 과적합
감소

▼ `alpha[default=0,
alias: reg_alpha]`

피쳐 개수가 많을
경우 적용 검토 → 값
이 클수록 과적합
감소

▼

`scale_pos_weight[de`

특정 값으로 치우친
비대칭한 클래스로
구성된 데이터 세트
의 균형을 유지하기
위한 파라미터

<과적합 문제가 심각한 경우>

- eta 값을 낮춤. → num_round(or n_estimators)는 반대로 높여줘야 함.
- max_depth 값을 낮춤.
- min_child_weight 값을 높임.
- gamma 값을 높임.
- subsample과 colsample_bytree를 조정 ⇒ 트리가 복잡하게 생성되는 것을 막을 수 있음.

XGBoost 자체적으로 교차 검증, 성능 평가, 피쳐 중요도 등의 시각화 기능을 가지고 있음.
또한 XGBoost는 기본 GBM에서 부족한 다른 여러 가지 성능 향상 기능이 있음.

- 조기 중단(Early Stopping) 기능: 수행 속도 향상

- 기본 GBM의 경우, n_estimators(or num_boost_rounds)에 지정된 횟수만큼 반복적으로 학습 오류 감소시키며 학습을 진행하면서 중간에 반복을 멈출 수 없고 지정된 횟수를 완료해야 함.

⇒ XGBoost, LightGBM은 모두 조기 중단 기능이 있어서 n_estimators에 지정한 부스팅 반복 횟수에 도달하지 않더라도 예측 오류가 더 이상 개선되지 않으면 반복을 끝까지 수행하지 않고 중지해 수행 시간을 개선할 수 있음.

파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

xgboost(XGBoost의 파이썬 패키지)는 자체적으로 교차 검증, 성능 평가, 피쳐 중요도 등의 시각화 기능을 가지고 있음. + 조기 중단 기능

- **위스콘신 유방암 데이터 세트:** 종양 크기, 모양 등 다양한 속성값을 기반으로 악성 종양 인지 양성 종양 인지를 구분한 분류 데이터 세트
 - 양성 종양: 비교적 성장 속도가 느리고 전이 x
 - 악성 종양: 주위 조직에 침입하면서 빠르게 성장하고 신체 각 부위에 확산 되거나 전이되어 생명 위협

코드 필사에서 이어서~

사이킷런 래퍼 XGBoost의 개요 및 적용

1. **XGBClassifier:** 분류를 위한 래퍼 클래스
2. **XGBRegressor:** 회귀를 위한 래퍼 클래스

기존 사이킷런에서 일반적으로 사용하는 하이퍼 파라미터와 호환성을 유지하기 위해 기존의 xgboost 모듈에서 사용하던 네이티브 하이퍼 파라미터를 변경

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

xgboost의 n_estimators & num_boost_round 하이퍼 파라미터는 =

if 두 개가 동시에 사용되면, 하이썬 래퍼 XGBoost API에서는 n_estimators 파라미터를 무시하고 num_boost_round 파라미터 적용

코드 필사에서 이어서~

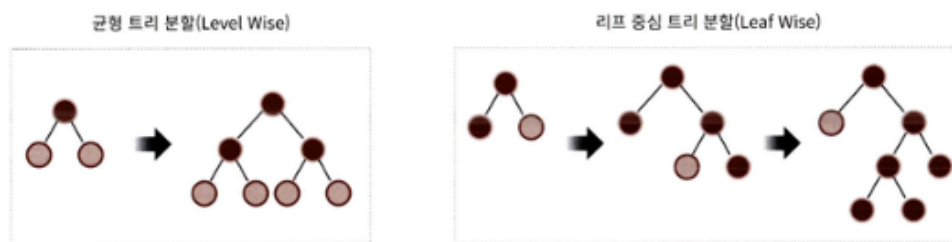
4.7 LightGBM

LightGBM의 XGBoost 대비장점

- 더 빠른 학습과 예측 수행 시간
- 더 적은 메모리 사용량
- 카테고리형 피처의 자동 변환과 최적 분할(원-핫 인코딩 등을 사용하지 않고도 카테고리형 피처를 최적으로 변환하고 이에 따른 노드 분할 수행)

▼ 리프 중심 트리 분할 방식 사용

결국은 균형 트리 분할 방식보다 예측 오류 손실을 최소화 할 수 있음.



LightGBM 설치

코드 필사에서 이어서~

LightGBM 하이퍼 파라미터



주요 파라미터

▼ num_iterations [default = 100]

- 반복 수행할 트리의 개수(부스팅 반복 횟수)
- 크게 하면 예측 성능이 좋아질 수 있지만, 너무 크면 과적합으로 성능이 떨어질 수 있음
- 사이킷런 GBM, XGBoost 사이킷런 래퍼의 n_estimators와 같은 파라미터
- LightGBM 사이킷런 래퍼(LGBMClassifier)에서는 n_estimators로 사용

▼ learning_rate [default = 0.1]

- 0~1 사이 값. 부스팅 스텝을 반복할 때마다 업데이트되는 학습률
- 보통 n_estimators를 크게, learning_rate를 작게 하면 성능이 좋아질 수 있음
- 단, 과적합 가능성과 학습 시간 증가도 함께 고려해야 함
- GBM, XGBoost의 learning_rate와 같은 파라미터

▼ max_depth [default = -1]

- 트리 기반 알고리즘의 max_depth와 같음
- 0보다 작은 값이면 깊이 제한 없음
- LightGBM은 Leaf wise 방식이라 Depth wise 트리보다 깊이가 상대적으로 더 깊음

▼ min_data_in_leaf [default = 20]

- 결정 트리의 min_samples_leaf와 같은 파라미터
- 리프 노드가 되기 위한 최소 레코드 수 → 과적합 제어용
- 사이킷런 래퍼(LGBMClassifier)에서는 min_child_samples로 사용

▼ num_leaves [default = 31]

- 하나의 트리가 가질 수 있는 최대 리프 개수

▼ boosting [default = gbdt]

- 부스팅 트리를 생성하는 알고리즘 지정
- gbdt: 일반적인 그래디언트 부스팅 결정 트리
- rf: 랜덤 포레스트

▼ bagging_fraction [default = 1.0]

- 트리가 커져서 과적합되는 것을 막기 위해 데이터를 샘플링하는 비율
- 사이킷런 GBM, XGBClassifier의 subsample과 같은 파라미터
- 사이킷런 래퍼(LGBMClassifier)에서는 subsample로 사용

▼ feature_fraction [default = 1.0]

- 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율. 과적합 방지용
- GBM의 max_features와 유사
- XGBClassifier의 colsample_bytree와 같음
- 사이킷런 래퍼(LGBMClassifier)에서는 colsample_bytree로 사용

▼ lambda_l2 [default = 0.0]

- L2 regularization 제어 값
- 피처 수가 많을 때 값을 키우면 과적합 감소 효과
- XGBClassifier의 reg_lambda와 같음
- 사이킷런 래퍼(LGBMClassifier)에서는 reg_lambda로 사용

▼ lambda_l1 [default = 0.0]

- L1 regularization 제어 값. L2처럼 과적합 제어용
- XGBClassifier의 reg_alpha와 같음
- 사이킷런 래퍼(LGBMClassifier)에서는 reg_alpha로 사용

Learning Task 파라미터

▼ objective

- 최솟값을 가져야 할 손실함수 정의. XGBoost의 objective와 같음
- 회귀, 다중 클래스 분류, 이진 분류 등 문제 유형에 따라 손실함수가 지정됨

하이퍼 파라미터 튜닝 방안

기본 튜닝 방안: num_leaves의 개수를 중심으로

min_child_samples(min_data_in_leaf), max_depth를 함께 조정하면서 모델의 복잡도를 줄이는 것

- num_leaves: 개별 트리가 가질 수 있는 최대 리프의 개수, LightGBM 모델의 복잡도를 제어하는 주요 파라미터

- **min_data_in_leaf**: 사이킷런 래퍼 클래스에선 min_child_sample, 과적합을 개선하기 위해 중요한 파라미터.
- **max_depth**: 명시적으로 깊이의 크기 제한/ num_leaves, min_data_in_leaf와 결합해 과적합을 개선하는 데 사용

파이썬 래퍼 LightGBM과 사이킷런 래퍼 XGBoost, LightGBM 하이퍼 파라미터 비교

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

LightGBM 적용 - 위스콘신 유방암 예측

코드 필사에서 이어서~

4.8 분류 실습 - 캐글 산탄데르 고객 만족 예측

: 370개의 피쳐로 주어진 데이터 세트 기반에서 고객 만족 여부를 예측하는 것.

산탄데르 은행이 캐글에 경연을 의뢰한 데이터로서 피쳐 이름은 모두 익명 처리돼 이름만을 가지고 어떤 속성인지는 추정할 수 x.

- 클래스 레이블 명: TARGET
 - 1: 불만을 가진 고객
 - 0: 만족 고객

코드 필사에서 이어서~

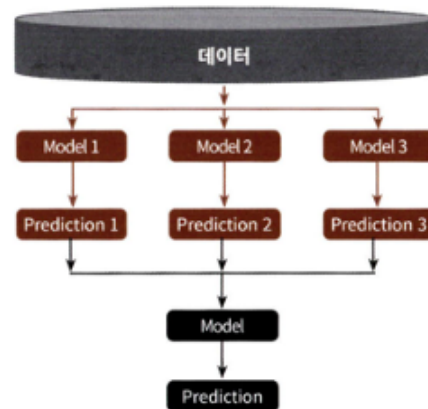
4.10 스택킹 앙상블

스택킹: 여러 알고리즘을 서로 결합해 예측 결과 도출 ← 배깅, 부스팅과 공통점

차이점: 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행한다는 것.

⇒ 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 절도의 ML 알고리즘으로 최종 학습을 수행하고 테스트 데이터를 기반으로 다시 최종 예측을 수행하는 방식 스택킹 모델은 두 종류의 모델이 필요함.

1. 개별적인 기반 모델
2. 이 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델



1단계: 개별 모델(Base Model) 학습

- $M \times N$ 데이터 세트로 여러 개(K개)의 서로 다른 모델을 각각 학습시킵니다.
- 예: KNN, 랜덤 포레스트, 결정 트리, 에이다부스트 등

2단계: 개별 모델별 예측

- 각 모델이 예측을 수행 → 모델마다 **M개 로우 × 1개 칼럼**의 예측 레이블 값이 나옵니다.

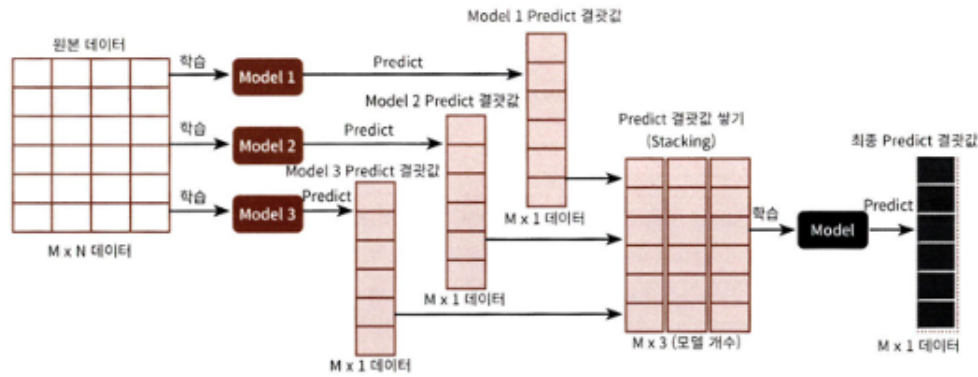
3단계: 예측값 스택킹 (새 데이터 세트 생성)

- K개의 예측 결과를 옆으로(칼럼 방향으로) 붙입니다.
- 결과: **$M \times K$** 형태의 새로운 데이터 세트
- 즉, 원본 피쳐(N개) 대신 "**모델들의 예측값**"이 새로운 피쳐가 됩니다.

4단계: 최종 모델(Meta Model) 학습 및 예측

- 스택킹된 $M \times K$ 데이터 세트를 입력으로, 원본 레이블을 정답으로 하여 최종 모델을 학습합니다.
- 이 최종 모델이 최종 예측을 수행합니다.

원본 데이터 → 여러 모델의 예측값 → 예측값들을 피쳐로 쌓은 새 데이터 → 메타 모델 → 최종 예측



기본 스택킹 모델

코드 필사에서 이어서~

CV 세트 기반의 스택킹

: 과적합 개선하기 위해 최종 메타 모델을 위한 데이터 세트를 만들 때 교차 검증 기반으로 예측된 결과 데이터 세트 이용.

⇒ 개별 모델들이 각각 교차 검증으로 메타 모델을 위한 학습용 스택킹 데이터 생성과 예측을 위한 테스트용 스택킹 데이터를 생성한 뒤 이를 기반으로 메타 모델이 학습과 예측을 수행

- STEP 1: 각 모델별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터 생성
- STEP 2: 1에서 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 학습할 최종 학습용 데이터 세트 생성. 마찬가지로 각 모델들이 생성한 테스트용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 예측할 최종 테스트 데이터 세트를 생성. 메타 모델은 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습한 뒤, 최종적으로 생성된 테스트 데이터 세트를 예측하고, 원본 테스트 데이터의 레이블 데이터를 기반으로 평가.

<CV 세트 기반 스택킹: 스텝 1 정리>

핵심 개념

- 개별 모델에서 **메타 모델용 학습 데이터와 테스트 데이터를 교차 검증으로 생성하는 것**이 핵심임.
- 스텝 1은 **개별 모델 단위**로 수행하며, 같은 로직을 모든 개별 모델에 똑같이 적용함.
- 스텝 2(메타 모델 학습)는 비교적 간단하므로, 여기서는 스텝 1을 중심으로 정리함.

전제

- 학습 데이터를 N개의 폴드로 나눔. 여기서는 **3개 폴드**로 가정함.

- 폴드가 3개이므로 같은 작업을 **3번 반복함**.

1회 반복 시 수행하는 작업

1. **학습**: 3개 폴드 중 2개는 학습용, 나머지 1개는 검증용으로 나눔. 학습용 2개 폴드로 개별 모델을 학습시킴.
2. **검증 폴드 예측**: 학습된 모델로 검증 폴드 1개를 예측하고 결과를 저장함.
3. **테스트 세트 예측**: 같은 모델로 원본 테스트 데이터를 예측하고 결과를 저장함.

반복별 폴드 구성

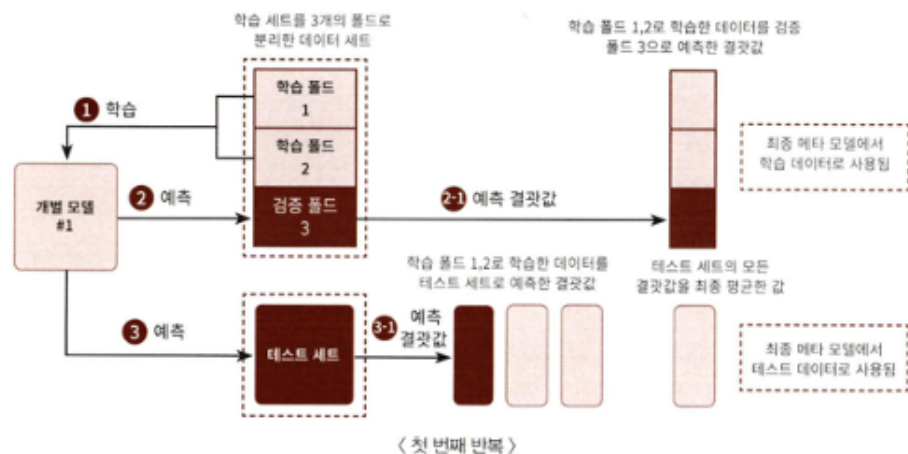
반복	학습 폴드	검증 폴드	결과
1회차	1, 2	3	폴드 3 예측값 + 테스트 세트 예측값 ①
2회차	1, 3	2	폴드 2 예측값 + 테스트 세트 예측값 ②
3회차	2, 3	1	폴드 1 예측값 + 테스트 세트 예측값 ③

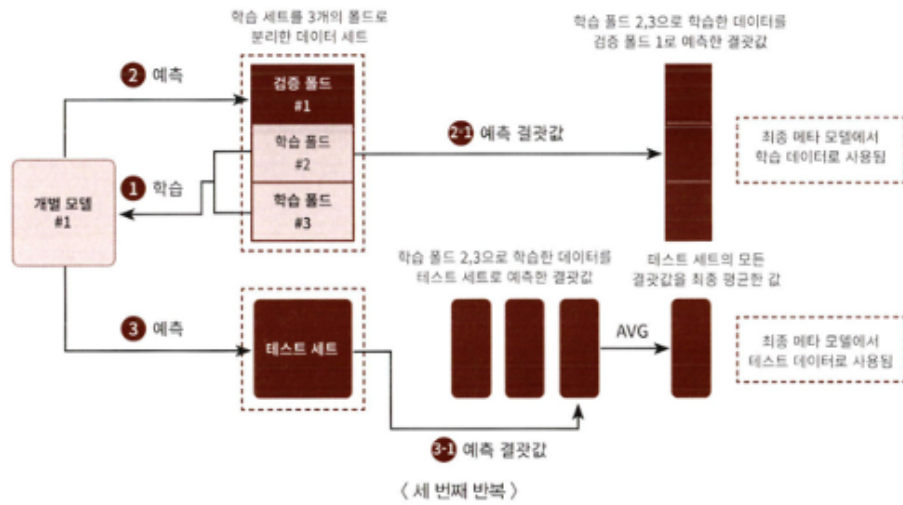
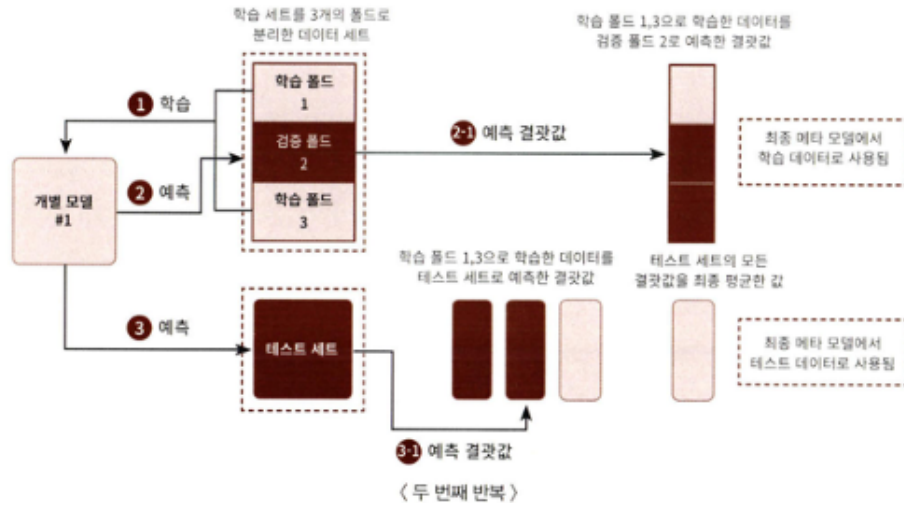
반복할 때마다 검증 폴드를 바꾸므로, 3번이 끝나면 **학습 데이터 전체가 한 번씩 예측됨**.

최종 결과물

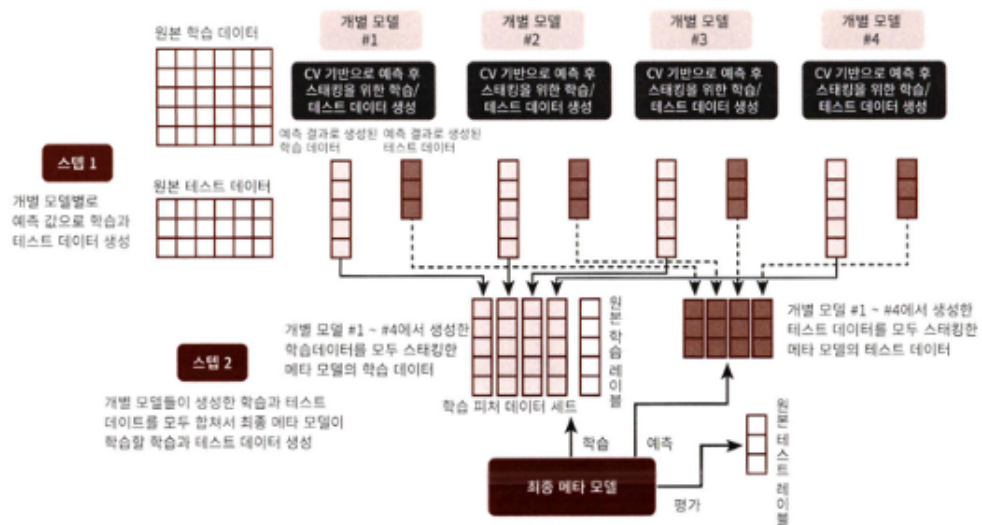
- **메타 모델의 학습 데이터**: 3번의 반복에서 나온 검증 폴드 예측값을 모두 합친 것. 학습 데이터 전체 크기와 같음.
- **메타 모델의 테스트 데이터**: 3번의 반복에서 나온 테스트 세트 예측값 3개를 **평균한 것**.

개별 모델마다 "2개 폴드로 학습 → 검증 폴드 예측 + 테스트 세트 예측"을 3번 반복함. 검증 폴드 예측값을 이어 붙이면 **메타 학습 데이터**가 되고, 테스트 예측값을 평균하면 **메타 테스트 데이터**가 됨.





< CV 기반의 스택킹 모델 전체를 도식화 >



4.11 정리

앙상블

- 분류에서 가장 많이 쓰이는 앙상블은 대부분 결정 트리 기반임.
- 여러 개의 약한 학습기(Weak Learner)를 결합해 변동성을 줄이고 예측 오류를 줄여 성능을 높임.

결정 트리

- 정보의 균일도를 기반으로 규칙 트리를 만들어 예측함.
- 장점: 직관적이어서 예측 결과가 어떻게 나왔는지 쉽게 알 수 있음.
- 단점: 균일한 결과를 위해 트리가 깊고 복잡해지면서 과적합이 쉽게 발생함.

배깅

- 학습 데이터를 중복 허용해 여러 세트로 샘플링하고, 각 세트를 약한 학습기가 학습한 뒤 결과를 결합함.
- 대표 알고리즘은 랜덤 포레스트. 수행 시간이 빠르고 예측 성능이 비교적 안정적임.

부스팅

- 현재 앙상블의 주류는 배깅보다 부스팅임.
- 학습기들이 순차적으로 학습하며, 예측이 틀린 데이터에 가중치를 부여해 다음 학습기가 더 정확히 예측하도록 함.
- GBM: 예측 성능은 뛰어나지만 수행 시간이 너무 오래 걸림.
- XGBoost: 캐글 대회 우승 알고리즘으로 불리며 명성을 쌓음.
- LightGBM: XGBoost보다 학습 시간이 빠르면서도 비슷한 성능을 보임.
- 두 패키지 모두 사이킷런 래퍼 클래스를 제공해, 다른 Estimator와 같은 방식으로 쉽게 사용할 수 있음.

스태킹

- 여러 개별 모델의 예측 데이터를 기반으로, 최종 메타 모델이 학습할 학습 데이터와 테스트 데이터를 다시 생성하는 기법임.
- 핵심은 메타 모델용 데이터 세트를 개별 모델의 예측값을 쌓는(stacking) 방식으로 만드는 것임.